

Apple Leaf Disease Platform Documentation

This document provides a comprehensive explanation of every file and folder used in the Apple Leaf Disease Platform project.

Project Overview

The Apple Leaf Disease Platform is built using three primary components:

1. **backend/**: A robust FastAPI server that acts as the core intelligence capable of processing image classification requests.
2. **website/**: A user-friendly Flask application providing an interactive web-based browser interface.
3. **mobile_app/**: A cross-platform Flutter mobile application designed for farmers out in the field.

1. Backend Directory (/backend)

- **main.py**:

The entry point for the FastAPI server. It initializes the app, defines all API endpoints (such as /detect for predicting diseases from images), handles incoming requests, loads the AI model during early startup, and links to the disease_database.py.

- **disease_database.py**:

A centralized Python module that stores a dictionary of all supported diseases, alongside detailed information for each (symptoms, organic treatments, chemical treatments, and preventive measures). Keeping this separate from main.py ensures the code stays clean and maintainable.

- **models/best.onnx**:

The pre-trained Machine Learning intelligence model weights. This file is parsed by OnnxRuntime to classify leaf images into the correct disease categorizations.

- **requirements.txt**:

A list of standard Python libraries required explicitly for the backend logic (like FastAPI for serving, uvicorn for hosting, ONNXRuntime for AI predictions, and Pillow for image processing).

2. Website Directory (/website)

- **app.py**:

The core Flask application file. It maps web URLs to specific HTML templates and acts as an intermediary bridge forwarding user-uploaded images from the browser to the backend FastAPI server, and then parsing

the predictions back.

- config.py:

Stores critical web server configurations such as secret session keys (for forms), file upload size limits, paths to temp directories, and the definitive URL endpoint where the backend API can be reached.

- requirements.txt:

Distinct from the backend requirements, this lists libraries used strictly for web serving (like Flask, Werkzeug, requests for calling the API, and WTForms for secure file uploads).

- run.py:

A simple development utility script to quickly spin up the Flask web server with predefined debug settings.

- templates/*.html:

A folder containing Jinja2 template files defining the UI (index.html, base.html, detect.html, etc.).

- static/css/ & js/:

Folders holding static assets to style the web pages and provide minor client-side interactions (like image preview before upload).

3. Mobile App Directory (/mobile_app)

- lib/main.dart:

The main entry point for the entirely separate Flutter mobile app. It configures app-wide settings like Dark/Light themes, Routing, and initializes core backend services.

- lib/screens/:

Contains individual UI page views of the mobile app (HomeScreen, CameraScreen, ResultsScreen).

- lib/services/api_service.dart:

Handles all outbound HTTP communication from the mobile phone reaching out to the FastAPI server.

- lib/widgets/:

Reusable micro UI components (custom buttons, cards, list tiles) to prevent code duplication across screens.

- lib/models/:

Defines typed Dart data classes (like DetectionResult.dart) ensuring the mobile app processes JSON data from the backend correctly and safely.

- pubspec.yaml:

The mobile app's package manager file governing all external Flutter packages (Camera dependencies, HTTP integrations) and defining font/image assets.

4. Root Project Files

- README.md:

Provides new developers with immediate setup and execution instructions for the entire platform from the root context.